

Gabarito e Explicações - Dia 3

Aquecimento

1) Lista Vazia e For

```
a = []
for i in range(5):
    num = int(input("Digite um número: "))
    a.append(num)
print(a)
```

Explicação: Iniciamos com os colchetes vazios. O laço com range(5) garante 5 repetições exatas, e a cada volta o método append() guarda o valor no final da estrutura.

2) Números Pares com Range

```
for i in range(0, 21, 2):
    print(i)
```

Explicação: A função range(início, fim, passo) é muito versátil. Definimos início 0, limite 21 (pois o último valor nunca é incluso, garantindo o 20) e passo 2.

3) Laço While Booleano

```
continuar = True
while continuar:
    palavra = input("Digite uma palavra: ")
    if palavra == "sair":
        continuar = False
```

Explicação: A variável continuar controla o fluxo. O while roda infinitamente até que a palavra seja "sair", momento em que atualizamos a variável para False, quebrando o laço na próxima verificação.

4) Modificando e Removendo

```
lista = [10, 20, 30, 40, 50]
lista[2] = 100
removido = lista.pop()
print(lista)
print("Removido:", removido)
```

Explicação: O índice 2 acessa o terceiro elemento (30). O método pop() atua no final da lista se nenhum índice for passado e retorna o valor ejetado para podermos salvá-lo.

5) Buscando na Cesta

```
cesta = ["Abacaxi", "Uva", "Romã", "Abacate"]
for fruta in cesta:
    if fruta == "Uva":
        print("Achei a Uva!")
    else:
        print(fruta)
```

Explicação: O for in percorre os elementos da lista diretamente. Usamos um simples if/else para interceptar a "Uva" e tratá-la de forma diferente das demais strings.

Avançando

6) Multiplicação de Lista e Índices

```
lista = [0] * 5
for i in range(len(lista)):
    lista[i] = i * 2
print(lista)
```

Explicação: [0] * 5 gera rapidamente [0, 0, 0, 0, 0]. O laço range(len(lista)) nos dá exatamente os índices de 0 a 4, permitindo modificar cada posição baseando-se no seu próprio índice.

7) Separador de Pares e Ímpares

```
pares = []
impares = []
while True:
    n = int(input("Número: "))
    if n == 0:
        break # Alternativa: mudar uma variável booleana
    if n % 2 == 0:
        pares.append(n)
    else:
        impares.append(n)
print("Pares:", pares)
print("Ímpares:", impares)
```

Explicação: O while True cria um loop infinito clássico, quebrado quando o usuário digita 0. O módulo % 2 destina cada número à lista correta usando append().

8) Manipulação Específica e Ordenação

```
numeros = [5, 2, 9, 1, 5, 6]
numeros.remove(5)
numeros.insert(2, 10)
```

```
numeros.sort()
print(numeros)
```

Explicação: remove(5) elimina apenas o primeiro '5' da esquerda para a direita. insert(2, 10) empurra os valores a partir do índice 2. Finalmente, sort() coloca tudo em ordem crescente definitivamente.

9) Fatiamento de Strings

```
nome = input("Digite seu nome: ")
pedaco = nome[0:4]
tamanho = len(nome)
print("Prefixo:", pedaco)
print("Total de caracteres:", tamanho)
```

Explicação: O slice [0:4] extrai os índices 0, 1, 2 e 3 (o 4 é exclusivo). Strings suportam fatiamentos idênticos aos de listas, e o len() conta todos os espaços e letras.

10) Concatenação e Inversão de Lista

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]
nova_lista = lista1 + lista2
nova_lista.reverse()
print(nova_lista)
```

Explicação: O operador + em listas não faz soma matemática, ele agrupa os elementos (concatena). O reverse() atua "in-place", ou seja, altera a própria lista diretamente.

Desafios

11) Fatiando Inteiros com Divisão

```
num = int(input("Digite um número: "))
digitos = []
while num > 0:
    ultimo_digito = num % 10
    digitos.append(ultimo_digito)
    num = num // 10
print(digitos)
```

Explicação: O módulo % 10 sempre isola o último dígito de um número. A divisão inteira // 10 o "corta" fora para a próxima rodada do laço. Isso fatora um número de trás para frente.

12) Inversão Manual de Lista

```
lista = []
for i in range(6):
    lista.append(int(input("Valor: ")))
```

```
lista_invertida = []
for i in range(-1, -7, -1):
    lista_invertida.append(lista[i])
print(lista_invertida)
```

Explicação: Aproveitamos o suporte do Python a índices negativos. O laço começa no -1 (último elemento) e vai até o -6 através do passo negativo -1. Cada elemento lido de trás pra frente é adicionado na nova lista.

13) Sequência de Fibonacci Dinâmica

```
lista = [0, 1]
limite = int(input("Quantidade de termos: "))
while len(lista) < limite:
    soma = lista[-1] + lista[-2]
    lista.append(soma)
print(lista)
```

Explicação: Controlamos a parada do laço pelo tamanho da lista len(lista). Lemos os últimos valores gerados pelos índices negativos dinâmicos -1 e -2 e os adicionamos sucessivamente.

14) Gerador de Múltiplos Primos

```
limite = int(input("Ver primos até: "))
primos = []
for num in range(2, limite + 1):
    achou_divisor = False
    for mult in range(2, num):
        if num % mult == 0:
            achou_divisor = True
            break
    if not achou_divisor:
        primos.append(num)
print(primos)
```

Explicação: O laço externo define qual número está sendo testado e o interno o testa contra todos os seus antecessores. Se encontrarmos um divisor exato, achou_divisor vira True. Os que resistirem a isso vão para a lista.